UNIVERSITÉ LIBRE DE BRUXELLES (ULB)



DATA WAREHOUSES (INFO-H419) PROJECT REPORT

BENCHMARKING AND QUERY OPTIMIZATION OF TPC-DS ON POSTGRESQL

ACADEMIC YEAR 2026 – 2027

Group Members: Xuân Hùng Lê – [000647865]
Ooi Kah Choo – [000647847]
Negar Sourati – [000648329]
Yiqing Kong – [000648010]

Instructor: Prof. Esteban Zimányi

Contents

1 Introduction and Technology Fundamentals **5**
- 1.1 Data Warehouses and Decision Support Systems . . . 5
- 1.2 Project Scope and Objectives . . . 5
- 1.3 PostgreSQL as an Analytical Data Warehouse . . . 6

2 Benchmarking with TPC-DS **7**
- 2.1 Overview of TPC-DS . . . 7
- 2.2 Logical Database Design . . . 7
- 2.3 Scaling Model and Data Generation . . . 9
- 2.4 Query Workload and Characteristics . . . 9
- 2.5 Benchmarking Methodology . . . 10

3 Implementation Details and Automation Pipeline **11**
- 3.1 Experimental Environment . . . 11
  - 3.1.1 Containers . . . 11
  - 3.1.2 Scale Factors, Databases and Scenarios . . . 12
  - 3.1.3 PostgreSQL Configuration . . . 12
  - 3.1.4 Effect of the Platform . . . 12
- 3.2 Data and Query Generation . . . 13
  - 3.2.1 Data Generation . . . 13
  - 3.2.2 Query Generation . . . 14
- 3.3 Physical Design: Indexes and Clustering . . . 14
  - 3.3.1 Finding Candidate Indexes . . . 15
  - 3.3.2 Selected Indexes . . . 15
  - 3.3.3 Clustering . . . 15
- 3.4 Optimizer Statistics, Load Test, Power Test and Benchmark Runs . . . 16
  - 3.4.1 Optimizer Statistics . . . 16
  - 3.4.2 Load Test . . . 17
- 3.5 Power Test and Benchmark Runs . . . 17
- 3.6 Benchmark Runs . . . 17

4 Results and Discussion **18**
- 4.1 Load Test . . . 18
- 4.2 Baseline Query Performance . . . 19
  - 4.2.1 Power Test . . . 19
  - 4.2.2 Choosing the Maximum Scale Factor . . . 19
- 4.3 Execution Time Distribution and Outlier Detection . . . 20
  - 4.3.1 Scale 1 . . . 20
  - 4.3.2 Scale 2 . . . 21
  - 4.3.3 Scale 5 . . . 21
  - 4.3.4 Scale 10 . . . 22
- 4.4 Query Execution Time Patterns Across Scale Factors . . . 23
- 4.5 Query Optimization . . . 25
  - 4.5.1 Avoiding repeated calculations. . . . 26
  - 4.5.2 Combining calculations across periods. . . . 27
  - 4.5.3 Pre-aggregating intermediate results. . . . 28
  - 4.5.4 Filtering early and restricting joins. . . . 30

4.5.5 Simplifying join syntax and predicates. 32
4.5.6 Queries combining multiple optimization techniques. 34
4.6 Indexes 36
4.6.1 Secondary Indexes 37
4.6.2 Clustering 37

5 Conclusion and Future Work **38**
5.1 Conclusion 38
5.2 Future Work 39
5.3 LLM Usage 39

Abstract

This report presents a performance evaluation and optimization study of a TPC-DS-derived analytical workload on PostgreSQL 16.15. Using a containerized setup and an automated benchmarking pipeline, we evaluate 99 analytical queries across four scale factors (SF $\in \{1, 2, 5, 10\}$, corresponding to roughly 1 GB to 10 GB of raw data). Baseline measurements and execution plan analysis show that while nearly 70% of the queries scale linearly with data size, total execution time is dominated by a small subset of outlier queries caused by multi-way self-joins over materialized Common Table Expressions (CTEs) and correlated subqueries. To address these bottlenecks, we rewrite 20 slow queries across the four scale factors, check output equivalence against the original queries, and re-benchmark the workload. We then evaluate the additional impact of secondary B-Tree indexes and physical table clustering. In our experiments, SQL query rewriting provides the largest improvement, reducing the reported SF = 10 workload total by 98.2% between the no-timeout baseline and the rewritten suite. Adding secondary indexes and physical clustering reduces the runtime of the optimized suite by up to another 22.3%, though at the expense of longer initial load times.

1 Introduction and Technology Fundamentals

1.1 Data Warehouses and Decision Support Systems

Modern organizations collect large volumes of operational data every day through transactional systems such as point-of-sale terminals, e-commerce platforms, and inventory logs. While Online Transaction Processing (OLTP) databases are built to handle fast, concurrent row-level inserts and updates while keeping only current state records, they are poorly suited for complex historical analysis. Running heavy multi-table aggregations directly on operational databases slows down daily transactions and cannot easily combine records across separate departments.

To separate analytical workloads from daily operations, enterprises consolidate historical records into a dedicated data warehouse [11]. Through Extraction, Transformation, and Loading (ETL) pipelines, data from different operational sources is cleaned, standardized into a common format, and appended into a central repository as time-stamped snapshots. Because analytical records are rarely modified once loaded, the database can organize tables around major business entities (such as customers, products, and sales channels) rather than individual transactional forms. In practice, organizations may build a central enterprise warehouse first and derive smaller departmental subsets, known as data marts, or build departmental marts first and connect them through shared dimensions [11].

At the logical level, analytical schemas typically organize data into fact tables and dimension tables. Fact tables store quantitative measurements of business events (such as sales amounts, quantities, or return losses) and grow rapidly over time, while dimension tables store descriptive context (such as dates, store locations, customer demographics, and item hierarchies). When dimension tables are normalized into sub-dimensions or shared across multiple fact tables, the resulting structure forms a snowflake or constellation schema.

Data warehouses provide the storage foundation for Decision Support Systems (DSS), which help analysts and managers evaluate business trends and answer semi-structured planning questions [10]. Unlike routine operational lookups that retrieve a single record by primary key, decision support workloads combine scheduled summary reports with unpredictable ad-hoc queries. These queries scan millions of historical fact rows, join multiple dimensions, compare metrics across time periods, and compute rankings or statistical aggregates. Because a single poorly planned query can take hours to finish, evaluating how a database management system plans and executes decision support workloads is a central problem in data warehouse engineering.

1.2 Project Scope and Objectives

When analytical queries slow down as data volume grows, database engineers generally have two ways to improve performance: rewriting the SQL queries so the optimizer can choose a more efficient execution plan, or changing the physical database design by adding secondary indexes and reordering table storage. However, testing these changes on a realistic workload requires a controlled benchmark environment. A rewrite that speeds up a query is only valid if it returns the exact same results as the original query, and an index that speeds up one query may slow down another or add substantial overhead during data loading.

The goal of this project is to study how an open-source relational database handles a complex decision support workload as data size increases across several scale factors, identify the main execution bottlenecks, and measure the separate and combined effects of SQL query rewriting, secondary B-Tree indexing, and physical clustering.

1.3 PostgreSQL as an Analytical Data Warehouse

We conduct our study on PostgreSQL, an open-source object-relational database system widely used in both transactional and analytical applications [8, 2]. Although PostgreSQL stores tables in a traditional row-oriented format rather than a specialized columnar layout, it includes several features needed for decision support workloads [1, 3]:

- **Analytical SQL Support:** PostgreSQL supports Common Table Expressions (`WITH` queries, with optional `MATERIALIZED` or `NOT MATERIALIZED` hints), window functions (`OVER (PARTITION BY ...)`), filtered aggregates (`FILTER (WHERE ...)`), hierarchical grouping (`ROLLUP` and `GROUPING()`), and standard set operations (`UNION`, `INTERSECT`, and `EXCEPT ALL`).
- **Cost-Based Query Optimizer:** Before executing a query, the planner uses column statistics gathered by `ANALYZE` (such as row counts, most common values, null fractions, and histograms) to estimate the cost of candidate execution trees [5]. It chooses between sequential scans, bitmap scans, and index scans, and selects among nested loop, hash, and sort-merge join algorithms.
- **Memory Management and Parallel Query Processing:** Hash tables for joins and aggregations, as well as sorting buffers for window functions and `ORDER BY` clauses, are allocated within per-operation working memory (`work_mem`), while frequently accessed table pages are cached in shared memory (`shared_buffers`). When intermediate results exceed `work_mem`, PostgreSQL spills temporary batches to disk. In addition, the execution engine can launch parallel background workers to speed up large sequential scans, hash joins, and partial aggregations.
- **Plan Inspection and Extensibility:** PostgreSQL provides detailed runtime diagnostics through `EXPLAIN (ANALYZE, BUFFERS)`, which reports actual row counts, loop iterations, execution times, and buffer reads for every node in the plan tree. It also supports extensions such as `HypoPG`, which allows index advisers to test hypothetical B-Tree indexes inside the planner's cost model without writing index structures to disk.

Because PostgreSQL's optimizer relies on single-table statistics and assumes independence between columns by default, complex analytical queries with chained Common Table Expressions, multi-column joins, or correlated subqueries can cause large row-count estimation errors. This makes PostgreSQL a practical platform for studying how query structure and physical indexing interact across growing data scales.

2 Benchmarking with TPC-DS

This chapter introduces the TPC-DS benchmark, its schema and its queries, and then how its data is scaled and how a benchmark is normally run. The setup we built on PostgreSQL is described in Chapter 3, and the measurements and query optimizations are reported in Chapter 4.

2.1 Overview of TPC-DS

TPC-DS is a decision-support benchmark maintained by the Transaction Processing Performance Council (TPC). Version 4.0.0 defines a retail data warehouse, a parameterized analytical workload, data-maintenance functions, execution rules, and performance metrics for evaluating a system under test (SUT) [9]. The business model is a product supplier selling through physical stores, catalogues, and the Web. It provides a realistic setting for common analytical tasks without representing one particular retailer.

To support benchmark preparation, TPC-DS provides two main utilities: `dsdgen`, which generates the synthetic data for the warehouse tables, and `dsqgen`, which generates executable SQL queries from the benchmark's query templates using substitution parameters [9].

The benchmark exercises large fact-table scans, joins to multiple dimensions, multi-level aggregation, complex subqueries, and both single-stream and concurrent analytical processing. Its design literature emphasizes that TPC-DS was intended to represent a broader and more varied decision-support workload than earlier benchmarks with fewer tables and queries [4, 6]. This diversity is relevant to query optimization: a rewrite that helps one access pattern may be neutral or harmful for another, and a plan that is efficient at one data scale need not remain so at a larger scale.

2.2 Logical Database Design

TPC-DS uses several related snowflake schemas rather than a single star. Its official logical design contains 24 base tables: seven fact tables and 17 dimension tables [9]. The benchmark models three retail sales channels: physical stores, catalogs, and the Web. Six fact tables form sales/returns pairs for these three channels, while the seventh records inventory. Dimensions are shared where the underlying business concept is common and are channel-specific where the sales process differs.

Fact rows refer to dimensions through surrogate-key columns. Shared date, item, customer, promotion, and demographic structures support cross-channel analysis, while channel-specific dimensions preserve details such as stores, call centres, catalogue pages, Web pages, and Web sites. Queries can therefore contain long join paths, multiple aliases of one dimension, and cross-channel set operations.

Figure 1 gives a simplified overview of the 24 base tables. The PostgreSQL schema declares primary keys but no foreign-key constraints, so the connecting lines show the logical key relationships defined by TPC-DS, not constraints stored in the database.

The schema has non-uniform scaling behavior. Large fact-table cardinalities grow with the scale factor, whereas several dimensions remain fixed or grow sub-linearly [4, 9]. Increasing the scale can thus change both the amount of data processed and the relative costs estimated by the PostgreSQL planner. Performance at SF1 should not be extrapolated by assuming that every table and every physical operator scales linearly.

Table 1: Logical organization of the TPC-DS base tables.

Group	Tables and analytical role
Sales facts	`store_sales`, `catalog_sales`, and `web_sales`; line-level sales activity for each channel.
Returns facts	`store_returns`, `catalog_returns`, and `web_returns`; returned products, quantities, and monetary measures.
Inventory fact	`inventory`; item quantities by warehouse and date.
Shared dimensions	`customer`, `customer_address`, `customer_demographics`, `household_demographics`, `income_band`, `date_dim`, `time_dim`, `item`, `promotion`, `reason`, `ship_mode`, and `warehouse`.
Channel dimensions	`store`, `call_center`, `catalog_page`, `web_page`, and `web_site`.

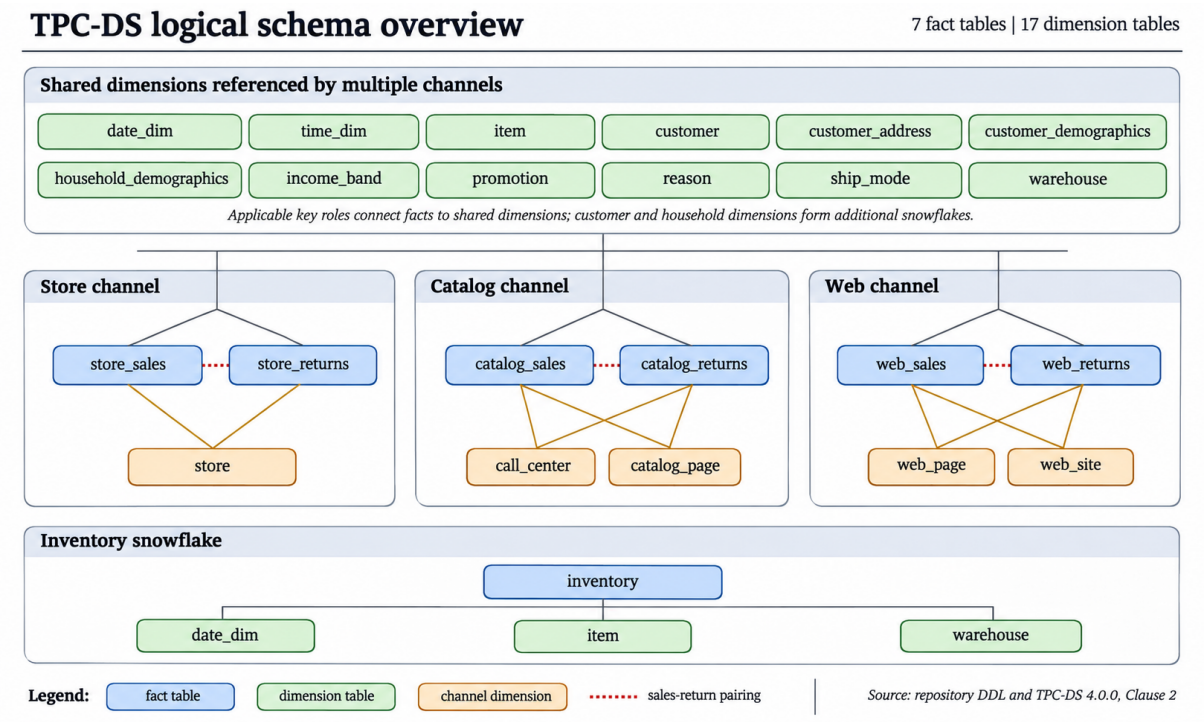


Figure 1: Simplified logical overview of the TPC-DS warehouse. Blue boxes represent fact tables, green boxes represent shared dimensions, and amber boxes represent channel-specific dimensions. Dashed lines pair each sales fact with its corresponding returns fact. The inventory fact forms a separate snowflake with date, item, and warehouse dimensions.

2.3 Scaling Model and Data Generation

TPC-DS uses the `dsdgen` utility to generate synthetic data for the tables defined in its warehouse schema. The generated data represents the different business activities included in the benchmark and provides a consistent dataset for evaluating database systems. The amount of data generated is controlled by a parameter called the scale factor (SF). This parameter is unitless and is approximately related to the number of gigabytes of raw generated data. However, it should not be interpreted as the actual storage size of the resulting database, since physical storage also depends on factors such as data types, page layout, indexes, and other database objects [9].

The official TPC-DS Version 4.0.0 specification defines the performance-test scale factors as 1,000, 3,000, 10,000, 30,000, and 100,000, corresponding to nominal raw-data scales of 1, 3, 10, 30, and 100 TB, respectively. SF1 is defined as a qualification scale rather than a reportable performance scale. The much smaller scale factors used in this project are therefore suitable for studying how performance changes with data size. These are used as experimental configurations and should not be interpreted as official reportable TPC-DS performance scales [9].

Increasing the scale factor results in a larger generated dataset, allowing database performance to be examined under different data volumes. However, the growth in data volume does not necessarily translate into identical growth in every table. The actual number of records depends on the scaling rules defined for individual tables in the TPC-DS specification. Furthermore, the size of the generated raw data differs from the final database size after loading, particularly when indexes and other database structures are added. Therefore, the scale factor provides a common basis for controlling dataset size, while actual database storage and table cardinalities must be measured separately.

2.4 Query Workload and Characteristics

The standard defines 99 query templates. A template specifies a functional query structure and substitution parameters; `dsqgen` instantiates it as executable SQL for a selected stream. A numbered query therefore identifies a template, whereas an experimental SQL file is a generated instance containing concrete years, months, thresholds, and literals. A TPC-DS query may contain an ordered set of one or more SQL statements [9].

The workload includes:

- joins among large sales, returns, or inventory facts and multiple dimensions;
- grouped and conditional aggregates, `ROLLUP`, and statistical functions;
- correlated and uncorrelated subqueries, `EXISTS`, `IN`, and set operations such as `UNION ALL`;
- window functions, ranking, sorting, and top-N limits; and
- comparisons across channels, years, customers, or products.

The official specification permits templates to contain redundant or non-optimal SQL structures so that the workload exercises optimizer transformations [9]. These structures also provide opportunities for manual rewriting, but a rewrite must preserve the generated instance's semantics, including NULL behavior, duplicate multiplicities, aggregate inputs, output ordering where relevant, and all substitution parameters.

In our implementation, generated SQL instances are not identical across all scale factors. Consequently, cross-scale performance comparisons must account for differences in query substitutions as well as database size. The generation procedure and observed differences are discussed further in Chapter 3.

2.5 Benchmarking Methodology

An official TPC-DS benchmark run has several phases [9]. A *load test* builds the database and is timed. A *power test* runs the 99 queries one after another in a single stream. Two *throughput tests* run several query streams concurrently, and two *data maintenance tests* insert and delete data between them. The results are combined into one metric, QphDS@SF, which also depends on the audited size of the data.

Our objective is different: we want to evaluate the impact of SQL query optimization and indexing on query execution performance in PostgreSQL. We therefore use TPC-DS as a source of standardized analytical queries and datasets, but we do not reproduce the complete official procedure. We run the load test and the power test with the original queries, and then repeat the query runs with the optimized queries and with different physical designs, so that we can compare execution times before and after each change.

The experimental design, i.e. the scale factors, the scenarios and the implementation of the load and power tests, is described in Chapter 3, and the measurements are reported in Chapter 4.

3 Implementation Details and Automation Pipeline

This chapter describes how we turned the TPC-DS specification [9] into an executable benchmark on PostgreSQL 16. All experiments in Chapter 4 use the setup described here, so every result is based on the same data, queries, settings and scripts.

We first describe the test environment and how the data and queries are generated. We then explain the physical design with indexes and clustering, followed by the optimizer statistics, the load test and the power test. The scripts that automate the whole experiment, and the commands to run them, are described in the README of the repository

3.1 Experimental Environment

All reported measurements were run on a single laptop (Table 2). PostgreSQL and the TPC-DS tools run in Docker. Docker gave the team the same software versions and settings during development and testing, and lets anyone rebuild the setup with the scripts and the README of the repository.

Component	Specification
CPU / RAM / storage	Apple M5 (ARM) / 32 GB / 1 TB SSD
Operating system	macOS Tahoe 26.6
Docker limits	8 CPUs, 16 GiB memory, 1 GiB swap, 926.3 GiB disk
PostgreSQL	16.15 (Debian)
TPC-DS toolkit	4.0.0

Table 2: Test environment.

3.1.1 Containers

The setup uses two kinds of containers (Table 3):

- **Database container** `tpcds-postgres`: the official `postgres:16` image, left unmodified, with `shm_size` of 2 GB because Docker's 64 MB default limits parallel query workers. There is one container and one named volume per scale factor, so the database persists between runs.
- **Tools container** `tpcds-tools`: built from `Dockerfile.tpcds`, it compiles and runs `dsdgen` and `dsqgen` (toolkit 4.0.0). It is started only to generate data and removed afterwards.

We keep the two separate because the toolkit is C code that needs compilers and build tools, which do not belong in a database image, and so that the database under test stays a standard `postgres:16` release. The local test folder is mounted into both containers, so PostgreSQL can read the files written by the tools container directly with `COPY`.

Container	Image	Role
`tpcds-postgres`	`postgres:16`	Creates the schema, loads data, runs queries
tpcds-tools	`Dockerfile.tpcds`	Runs `dsdgen` and `dsqgen`

Table 3: Docker containers.

3.1.2 Scale Factors, Databases and Scenarios

We use four scale factors, 1, 2, 5 and 10 (roughly 1, 2, 5 and 10 GB of raw data), which represent progressively larger datasets. They allow us to investigate how query execution time changes with increasing data volume and whether the effectiveness of the optimizations depends on the size of the dataset. To find a practical upper limit, we first ran the original, unoptimized queries as a baseline. At SF10 the full set of 99 queries took approximately eight hours. Given this and the available computing resources, we chose SF10 as the maximum scale factor, which is large enough to show performance differences while keeping the experiment manageable.

Each scale factor is kept completely separate. It has its own database named `tpcds_sf<N>_test` (for example `tpcds_sf1_test` and `tpcds_sf10_test`), and its own `data/`, `queries/`, `optimized_queries/` and `results/` folders. Loads, indexes and results of different sizes therefore never mix.

For each scale factor we compare three scenarios: the original queries (baseline), the optimized queries, and the optimized queries on a database with additional physical design, namely indexes and, in a further step, clustering (Section 3.3). No query time limit is used in the reported runs, so long-running queries complete and their execution times are recorded.

3.1.3 PostgreSQL Configuration

PostgreSQL's defaults are conservative and sized for small transactional workloads; for example `work_mem` defaults to 4 MB and `shared_buffers` to 128 MB. TPC-DS queries scan and join very large tables, so we raised the memory and parallelism parameters in Table 4. The values are identical for all scale factors and queries, and none is tailored to a specific table, index or query. They are set when the PostgreSQL server starts, stored as defaults of the benchmark database, and passed again by the benchmark runner before every measured query, so every session uses the same values. The benchmark runner can also abort a statement after a time limit. The reported benchmark runs use no limit, so even the slowest queries complete and are measured.

3.1.4 Effect of the Platform

Docker on macOS runs the containers inside a Linux virtual machine, and the Apple M5 is an ARM processor, whereas published TPC-DS results come from x86 servers. Our absolute run times are therefore specific to this setup and are not compared with published results. All scale factors and configurations ran on the same machine with the same settings, so the comparisons between them remain valid.

Parameter	Value
`shared_buffers`	4 GB
`work_mem`	512 MB
`hash_mem_multiplier`	2
`effective_cache_size`	10 GB
`max_parallel_workers_per_gather`	4
`random_page_cost`	1.1 (SSD)
`jit`	off
`statement_timeout`	off

Table 4: PostgreSQL configuration.

3.2 Data and Query Generation

The official TPC-DS toolkit (version 4.0.0), which we compile in the tools container, contains everything needed to prepare the benchmark (Figure 2): the generator `dsdgen` for the data, the 99 query templates with the generator `dsqgen` for the queries, and the schema file `tpcds.sql` that defines the 24 tables.

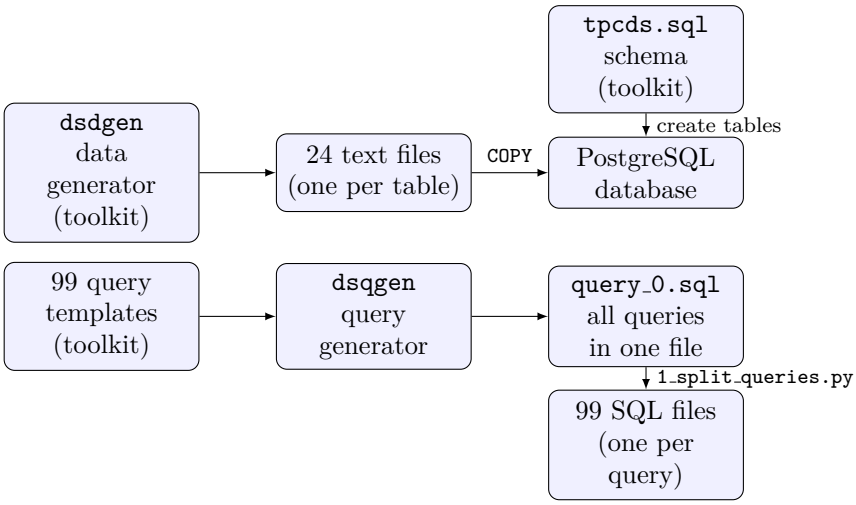


Figure 2: Preparation of data and queries.

3.2.1 Data Generation

`dsdgen` generates the synthetic data of the 24 tables described in Section 2.2. The scale factor chosen in Section 3.1 decides how much data it creates. For each scale factor it writes one plain text file per table into the `data/` folder of that scale factor.

To load these files, we first create the empty database of that scale factor with its 24 tables from the schema file `tpcds.sql`, and then load each text file into its table with PostgreSQL's `COPY` command. `COPY` reads a whole file into a table in one step, which is much faster than inserting the rows one by one with `INSERT`. Generating the files and creating the empty database are done once and are not timed; creating the tables, loading the data, creating the indexes and gathering statistics are part of the load test (Section 3.4). The exact commands are in the repository README.

3.2.2 Query Generation

The 99 TPC-DS queries are part of the toolkit as *templates*, not as finished SQL: each template has placeholders for values such as a year, a state or a product category. `dsqgen` fills the placeholders and writes the SQL in the syntax (*dialect*) of a chosen database system. The chosen values are not always the same for different scale factors, so we generated a separate set of 99 queries for each scale factor, and each scale factor's original queries are the reference for its rewritten queries in Chapter 4. A fixed random seed lets us regenerate the same set again for the same scale factor.

PostgreSQL is not among the supported dialects. We therefore based ours on Netezza, the closest one, and made a few corrections for PostgreSQL (below). `dsqgen` does not create one file per query: it writes all 99 queries one after another into a single file, `query_0.sql`. We split this file into 99 files with our own small script, `1_split_queries.py`, so that the benchmark runner can execute one query at a time.

Some generated queries failed on PostgreSQL. We corrected three categories in the templates before generation (Table 5):

1. **Date arithmetic (15 queries).** The templates add days with `date + 30 days`, which is invalid in PostgreSQL. Removing the word `days` gives `date + 30`, which PostgreSQL accepts for a date plus an integer.
2. **Column reference (Query 30).** The template uses `c_last_review_date_sk`, but our `customer` table defines the column as `c_last_review_date`.
3. **`lochierarchy` (Queries 36, 70, 86).** PostgreSQL does not accept the alias inside the `CASE` expression of `ORDER BY`. We replaced it with the underlying `grouping(...)` expressions.

These are minor query modifications (date expressions and aliases only), so the semantics of the queries do not change.

Issue	Fix applied	Queries
Invalid date arithmetic	Remove `days`: `date + 30`	5, 12, 16, 20, 21, 32, 37, 40, 77, 80, 82, 92, 94, 95, 98
Wrong column name	`c_last_review_date_sk` → `c_last_review_date`	30
`lochierarchy` alias	Replace with `grouping()` expressions	36, 70, 86

Table 5: Template modifications for PostgreSQL.

3.3 Physical Design: Indexes and Clustering

The base schema has no secondary indexes. Its only indexes are the primary-key indexes that PostgreSQL creates automatically for the primary-key constraints in `tpcds.sql`. A secondary index can speed up a query that filters or joins on its columns, but it costs disk space and slows down loading, so it has to be chosen with care. We benchmark three physical configurations on the same data (Table 6), so that the effect of the physical design can be separated from the effect of the scale factor.

Configuration	Description
`noindex`	Baseline: primary-key indexes only, no secondary indexes
`index`	Baseline plus the 8 secondary B-tree indexes of Table 7
`cluster`	Same indexes, plus the four fact tables physically reordered with `CLUSTER`

Table 6: Physical configurations.

3.3.1 Finding Candidate Indexes

With 99 queries and many columns, guessing indexes is unreliable, so we looked for tools that suggest them from the workload. PostgreSQL has no built-in index advisor, but it can be extended. We used two components:

- *HypoPG* is a PostgreSQL extension that creates *hypothetical* indexes. They exist only in the planner's cost model, so `EXPLAIN` shows the plan the optimizer *would* choose if the index existed, without spending disk space or build time.
- *Dexter* is an automatic indexer. It reads the query workload, proposes candidate indexes, uses HypoPG to compare the estimated cost with and without each candidate, and keeps those that lower the cost enough.

Dexter and HypoPG are only a selection aid: the cost estimates are the planner's, not measured times, and nothing hypothetical remains in the benchmarked databases.

3.3.2 Selected Indexes

Dexter and our review produced 14 candidate indexes. We then kept only simple indexes that stay close to the intent of TPC-DS, which restricts tuning structures such as indexes. Six candidates were removed: two combining two foreign keys (`store_sales`, `web_sales`), and four on `date_dim` and `item` that use plain attribute columns (`d_year`, `d_moy`, `i_category`, `i_current_price`, `i_manufact_id`). The remaining 8 indexes (Table 7) were created with `CREATE INDEX`, identically for all scale factors.

Table	Index columns
`store_sales`	`(ss_customer_sk)`, `(ss_sold_date_sk, ss_item_sk)`
`web_sales`	`(ws_bill_customer_sk)`, `(ws_sold_date_sk, ws_item_sk)`
`catalog_sales`	`(cs_sold_date_sk)`, `(cs_sold_date_sk, cs_bill_customer_sk)`, `(cs_sold_date_sk, cs_item_sk)`
`catalog_returns`	`(cr_returned_date_sk)`

Table 7: Indexes used in the `index` and `cluster` configurations.

3.3.3 Clustering

Some database systems have a "clustered index" that keeps the rows of a table sorted. PostgreSQL does not. Instead, the command `CLUSTER` rewrites a table once in the order of one index, so we speak of *clustering* a table. The `cluster` configuration therefore has

the same eight indexes as the `index` configuration, plus the physical reordering of the four fact tables, which we ran with `CLUSTER` each time on a date-based index (Listing 3.1). Rows with nearby dates are then stored next to each other, which helps queries that read a date range. `CLUSTER` is performed once as part of the load, before any query runs, and its time is included in the load time. PostgreSQL does not keep the order for later inserts, which is not relevant here because no data is modified after loading.

```
CLUSTER catalog_sales   USING idx_catalog_sales_cs_sold_date_sk;
CLUSTER catalog_returns USING idx_catalog_returns_cr_returned_date_sk;
CLUSTER store_sales     USING idx_ss_date_item;
CLUSTER web_sales       USING idx_ws_date_item;
```

Listing 3.1: Clustering the four fact tables.

3.4 Optimizer Statistics, Load Test, Power Test and Benchmark Runs

For every scale factor and configuration we first build the database (load test) and then run the 99 queries (Figure 3). The load test builds the database and measures how long this takes. The queries are then run and timed: once with the original queries as the power test, and three times with the optimized queries as benchmark runs.

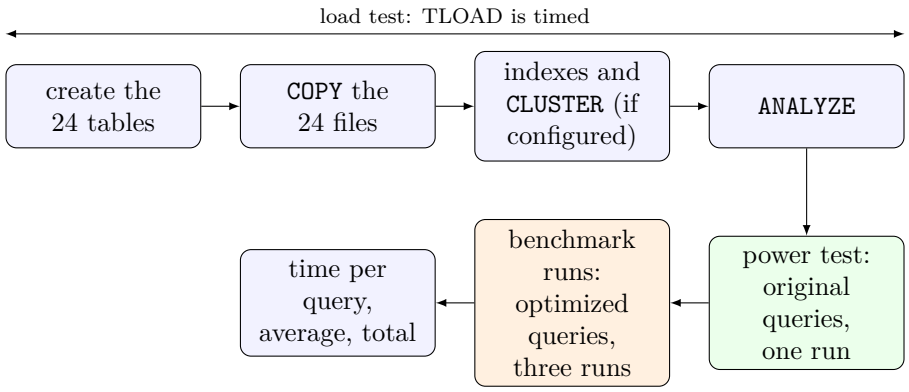


Figure 3: Load test, power test and benchmark runs for one scale factor and configuration. The power test uses the original queries and the noindex database; the benchmark runs use the optimized queries.

3.4.1 Optimizer Statistics

After loading and creating the indexes, we run `ANALYZE` on the whole database. `ANALYZE` samples every table and stores statistics in the system catalog: the number of rows, the number of distinct values and the distribution of values in each column. The query optimizer uses them to estimate how many rows each step of a query returns, and from that decides the join order, the join method and whether an index is worth using. Right after a bulk load these statistics do not exist yet, so the optimizer would work with default guesses and could choose poor plans, which would distort every timing. We therefore run `ANALYZE` as the last step of the load and count its time as part of the load.

3.4.2 Load Test

The load test measures how long it takes to make the database ready for the benchmark. Its duration, TLOAD, covers creating the tables, loading the data, creating the indexes and running `CLUSTER` where applicable, and gathering statistics. Generating the flat files with `dsdgen` and creating the empty database are not included. Each configuration of Table 6 is loaded independently into its own fresh database, with its own `COPY` of the data, so that the three loads cannot influence one another:

- `noindex`: tables, `COPY`, `ANALYZE`;
- `index`: tables, `COPY`, index creation, `ANALYZE`;
- `cluster`: tables, `COPY`, index creation, `CLUSTER` of the four fact tables, `ANALYZE`.

The timer starts immediately before the first `CREATE TABLE` and stops after `ANALYZE`. The PostgreSQL settings of Table 4 are applied to each database before the timer starts and verified again from a new connection before and after the load; the script stops if they differ. For every phase and every index the script also records its own duration, so the additional cost of the indexes and of `CLUSTER` can be read directly. The measured load times are reported in Section 4.1.

3.5 Power Test and Benchmark Runs

The power test follows the TPC-DS definition: it measures how fast PostgreSQL answers the 99 queries when they are run one after another in a single stream, with only one query running at a time. We run it once for each scale factor, with the original queries on the noindex database. This is our baseline. For every query we record the execution time, and the total time of a scale factor is the sum over the 99 queries. The queries run in template order rather than in random order, so that the scale factors stay comparable.

3.6 Benchmark Runs

All other measurements are benchmark runs of the optimized queries, on the noindex, index and cluster databases. They are executed in the same way, one query at a time, but each is run three times. For every query we report the average of the successful iterations, and the total time of a scale factor is the sum of these per-query averages. These runs are not power tests in the TPC-DS sense. We use them to compare the effect of query rewriting and of the physical design with the baseline.

4 Results and Discussion

In this chapter, we present the experimental results from running the TPC-DS benchmark on PostgreSQL 16.15 across SF $\in \{1, 2, 5, 10\}$. We first examine the database load times, baseline Power Test performance , outlier detection across scale factors, and query scaling patterns. We then present our SQL query optimizations, evaluate the impact of secondary indexes and table clustering, and summarize our main findings.

4.1 Load Test

Figure 4 illustrates the baseline database load time across different scale factors. The load test includes schema creation, loading all 24 TPC-DS tables using PostgreSQL COPY, and collecting statistics using ANALYZE. Data generation with dsdgen is excluded from the measurement. This experiment allows us to examine how the database loading time scales as the data volume increases.

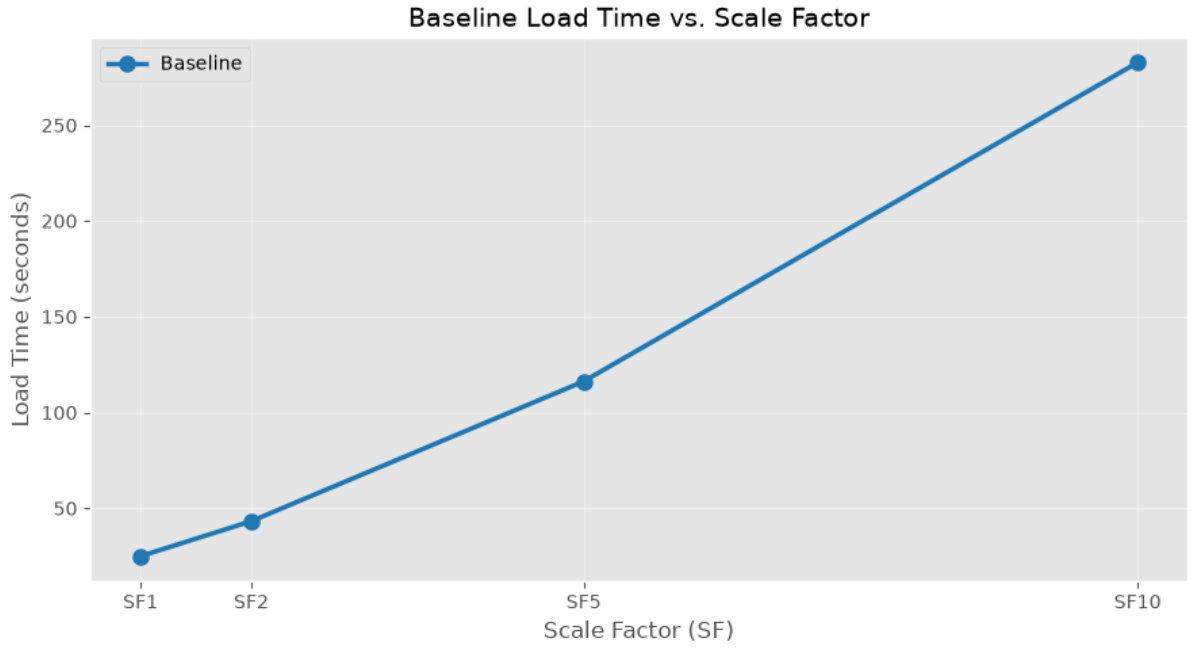


Figure 4: Database load time of the noindex configuration across different scale factors (SF1, SF2, SF5 and SF10).

Figure 5 compares the total database load times for three configurations. The noindex configuration (baseline) includes schema creation, data loading, and ANALYZE. In the load-test experiments shown in Figure 5, the index configuration creates 14 secondary indexes, while the cluster configuration creates 8 indexes and physically reorders 4 fact tables using CLUSTER. Each configuration is measured independently using a fresh database, with ANALYZE performed after all operations. By comparing these results, we evaluate the loading overhead associated with these different physical configurations across scale factors.

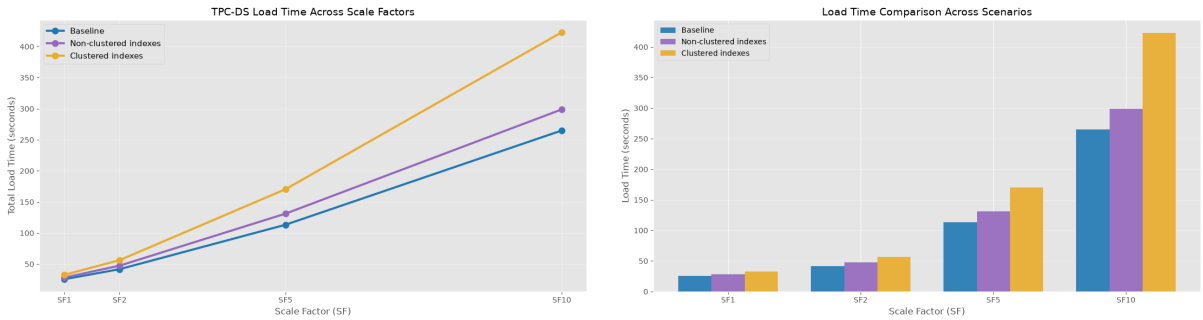


Figure 5: Comparison of database load times for the noindex, index and cluster configurations across different scale factors.

4.2 Baseline Query Performance

4.2.1 Power Test

The baseline experiments provide an initial evaluation of PostgreSQL query execution performance before applying any optimization techniques. In general, the overall execution time increases at larger scale factors, reflecting the growing computational cost of processing larger datasets. However, the total execution time at SF2 is slightly lower than at SF1, indicating that the relationship is not strictly monotonic across all tested scale factors.

You can see the overall execution time of 99 generated queries for our 4 scale factors in Figure 6

4.2.2 Choosing the Maximum Scale Factor

At SF10, executing the complete set of 99 queries required approximately eight hours Fig6. Considering this substantial execution time and the available computational resources, SF10 was selected as the maximum scale factor for the experiments. The baseline results also serve as a reference for evaluating the performance improvements achieved through query optimization and indexing in the following sections.

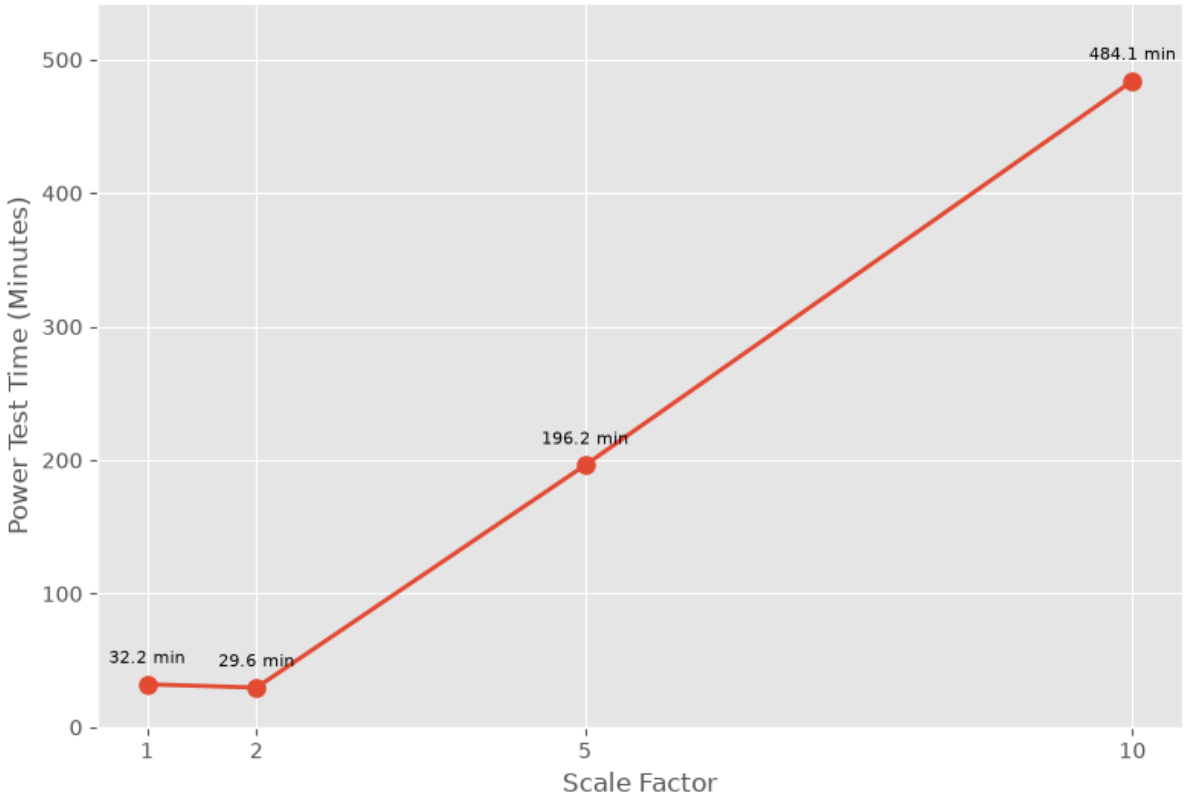


Figure 6: Overall Execution time of 99 queries for different scale factors

4.3 Execution Time Distribution and Outlier Detection

We use a logarithmic scale to visualize query execution times for each scale factor, because they vary significantly across queries. On a linear scale, queries with shorter execution times would appear compressed, making it difficult to distinguish their differences. Therefore, a logarithmic scale allows us to better visualize and compare the execution times of all queries. Additionally, we apply the Interquartile Range (IQR) method to the log-transformed execution times to identify outliers more effectively, as the execution times follow a highly skewed distribution.

While the boxplots in Figure 11 provide an overview of the execution time distributions and the presence of outliers, the individual bar charts presented in the following subsections highlight the specific queries identified as outliers using the same IQR method.

4.3.1 Scale 1

Execution times of the queries on scale factor 1 can be seen on Figure 7. The overall execution time of 99 queries is 1958.884 seconds (approximately 32 minutes). The outliers are Q1, Q4, Q6, Q11, Q14, Q74 and Q81.

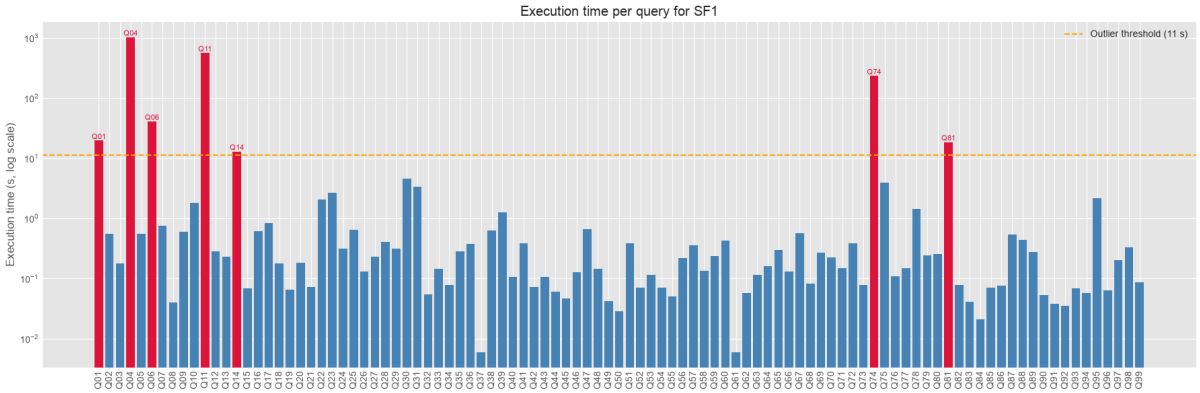


Figure 7: Execution time per query for SF1.

4.3.2 Scale 2

For SF2, the overall execution time of 99 queries is 1812.453 seconds(approximately 30 minutes). Most queries have relatively short execution times, while a few exhibit significantly higher values. According to the Barchart in Figure 8, we identified seven outliers Q1, Q4, Q6, Q11, Q14, Q74 and Q81.

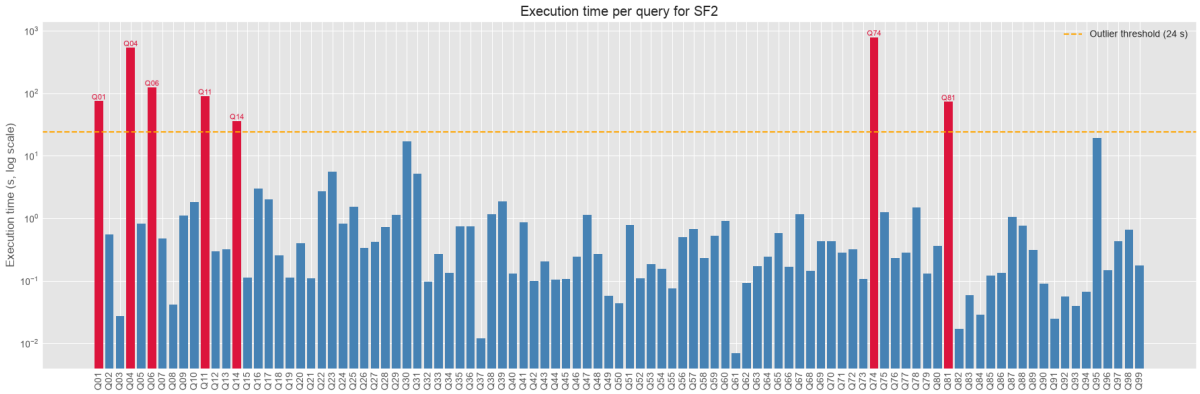


Figure 8: Execution time per query for SF2.

4.3.3 Scale 5

The result of the benchmark on SF 5 is shown in the Figure 9. The overall execution time of 99 queries is 11771.287 seconds (approximately 3.5 hours). The outliers are Queries Q1, Q4, Q6, Q11, Q14, Q30, Q74, Q81 and Q95.

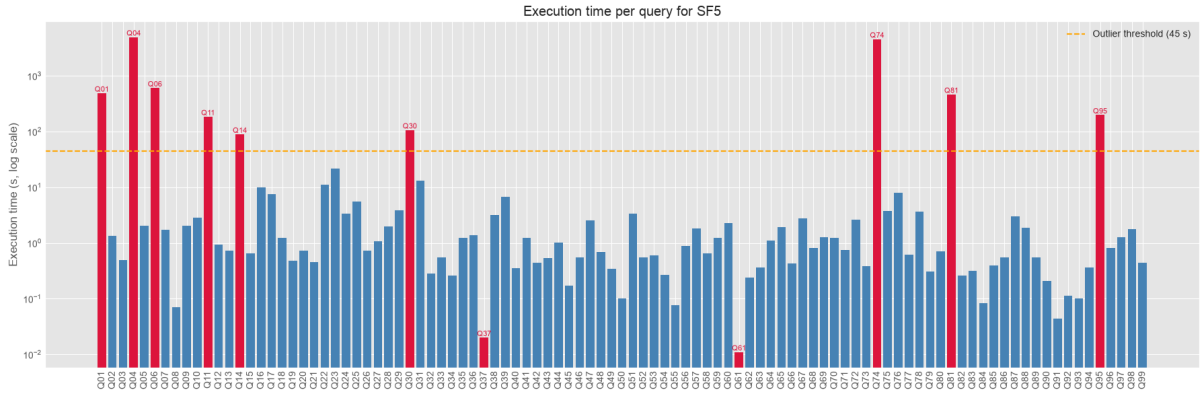


Figure 9: Execution time per query for SF5.

There are also two other outliers, such as Q37 and Q61 . Since our analysis focuses on queries with exceptionally long execution times, we do not consider these lower-end observations as relevant outliers. Therefore, only queries with unusually high execution times are considered outliers in this study.

4.3.4 Scale 10

Figure 10 shows the results for scale 10. Running the whole 99 queries took 29048.767 seconds(approximately 8 hours) The outliers are Queries Q1, Q4, Q6, Q11, Q14, Q30, Q74, Q81 and Q95. Query 74 itself took 4 hours to run successfully.

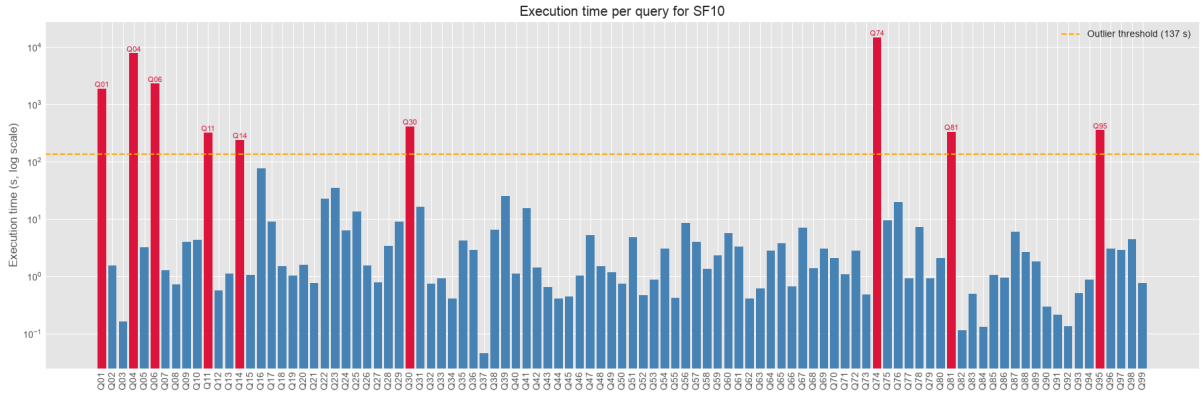


Figure 10: Execution time per query for SF10.

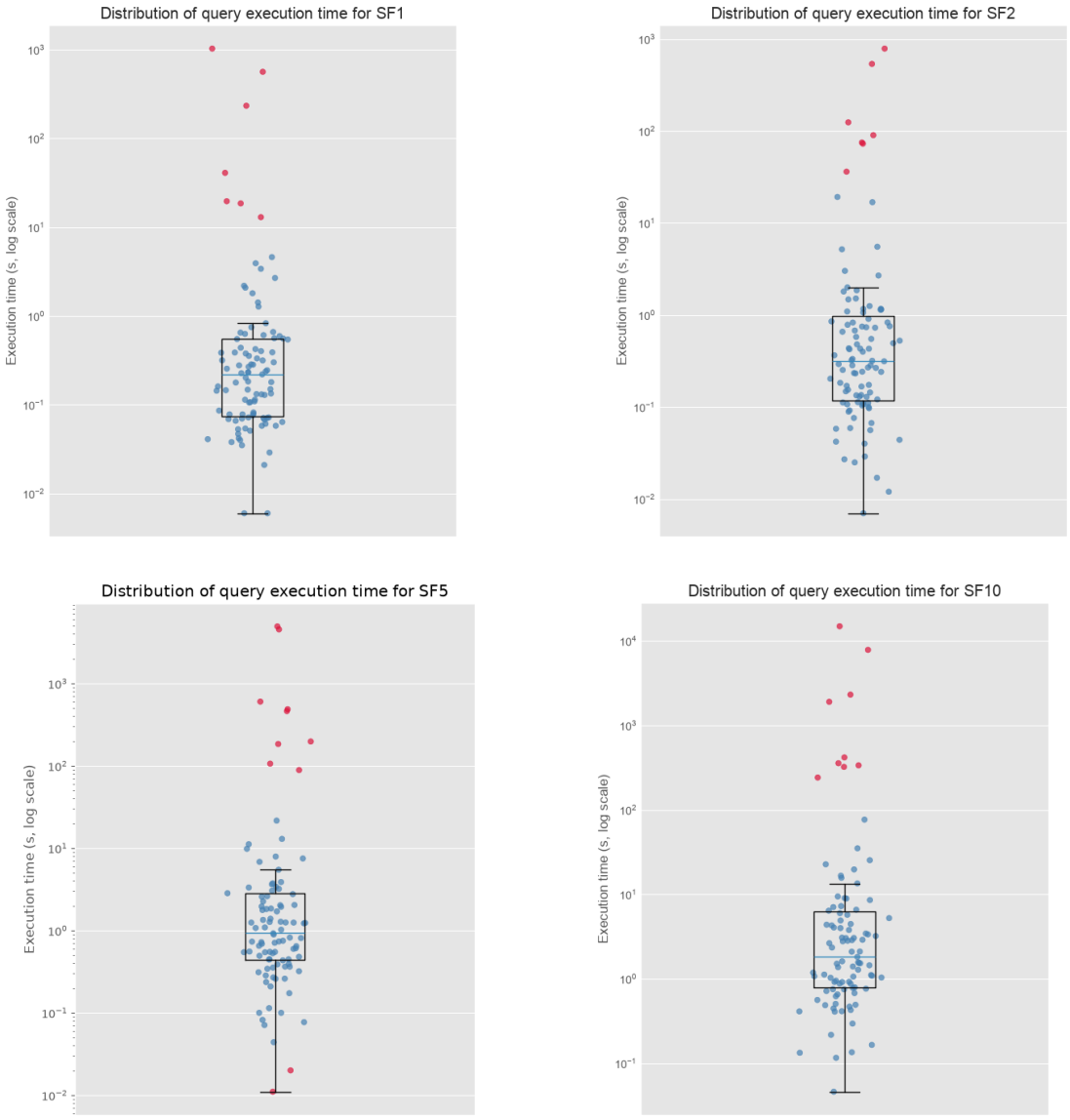


Figure 11: Distribution of TPC-DS query execution times across four scale factors (SF1, SF2, SF5, and SF10). The boxplots show the median, interquartile range, and variability of execution times, while red points highlight outliers. Execution times are displayed on a logarithmic scale.

4.4 Query Execution Time Patterns Across Scale Factors

To further analyze the relationship between query execution time and scale factor, we classified the scaling behavior of 99 queries into three categories: linear, exponential, and irregular. This classification was based on fitting linear and exponential models to the observed execution times across four scale factors (SF1, SF2, SF5, and SF10) and evaluating their goodness of fit using the coefficient of determination (R^2 score).

The R^2 score is calculated as follows [7]:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{1}$$

where y_i represents the observed query execution time, $\hat{y}_i$ is the execution time predicted by the fitted model, $\bar{y}$ is the mean of the observed execution times, and $n = 4$ corresponds to the four scale factors (SF1, SF2, SF5, and SF10).

An R^2 value closer to 1 indicates a better fit between the model and the observed execution times. We use a minimum threshold of $R^2 = 0.90$ to identify well-fitting models. A query is classified as exponential when its exponential model achieves an R^2 score at least 0.05 higher than the linear model. Queries that do not satisfy the classification criteria or exhibit significant decreases in execution time are classified as irregular.

The results show that 69 queries (69.7%) exhibited approximately linear growth, meaning that their execution times increased roughly proportionally to the scale factor, as illustrated in Figure 12. Furthermore, 9 queries (9.1%) followed exponential-like patterns, showing a more rapid increase in execution time as the data size grew, with representative examples presented in Figure 13. Finally, 21 queries (21.2%) demonstrated irregular behavior, exhibiting unexpected variations in execution time across different scale factors, as shown in Figure 14.

Overall, these findings indicate that most queries scale approximately linearly with increasing data size, while a smaller proportion experience more rapid growth or unpredictable performance.



Figure 12: 3 of 69 queries whose execution time follows linear

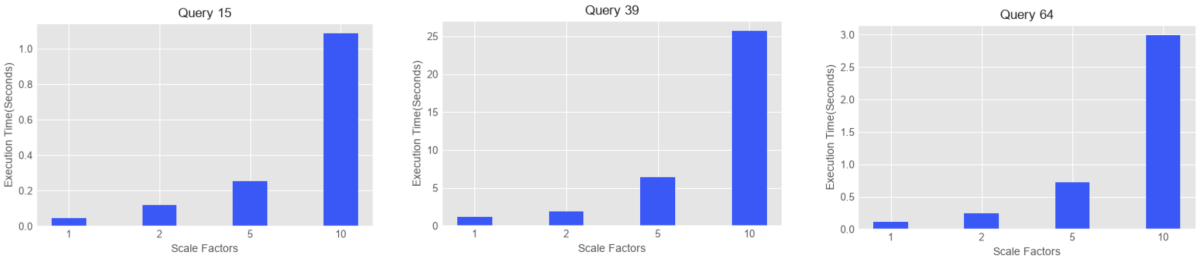


Figure 13: 3 of 9 queries whose execution time follows exponential

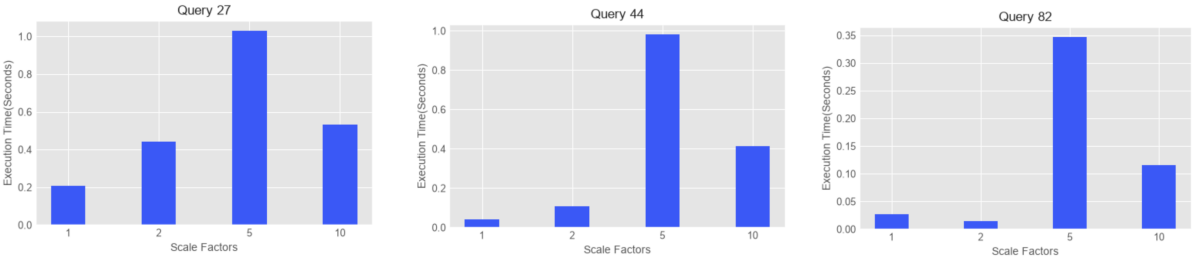


Figure 14: 3 of 21 queries with irregular execution time distribution

4.5 Query Optimization

The SQL examples and detailed explanations in this section primarily refer to SF10 query instances. The specific substitution parameters and aggregate functions may differ across scale factors, while the underlying optimization principles remain similar.

The 20 slowest queries identified in the SF10 baseline were selected for optimization. Based on the changes made to their SQL statements, we grouped the optimizations into five main categories: avoiding repeated calculations, combining calculations across periods, pre-aggregating intermediate results, filtering early and restricting joins, and simplifying join syntax and predicates. Each query is assigned to the category that best describes its main optimization, although some queries combine multiple techniques.

Table 8: Main SQL optimization strategies applied to the 20 selected queries.

Strategy	**Queries**	**Main idea**
Avoiding repeated calculations	Q1, Q6, Q23, Q30, Q41, Q81	Calculate intermediate results once, replace repeated correlated calculations, or simplify existence checks.
Combining calculations across periods	Q4, Q11, Q31, Q74, Q75	Calculate values for different years or quarters together using conditional aggregation, instead of joining separate results.
Pre-aggregating intermediate results	Q14, Q22, Q76	Aggregate large tables earlier to create smaller intermediate results for subsequent operations.
Filtering early and restricting joins	Q17, Q25, Q29, Q39, Q95	Reduce the rows involved in expensive joins and aggregations by filtering data earlier or restricting join inputs.
Simplifying join syntax and predicates	Q16	Replace implicit comma joins with explicit `JOIN` clauses, remove duplicate filter literals, and streamline `EXISTS` subqueries.

Several optimized queries use Common Table Expressions (CTEs) to organize intermediate calculations. A CTE, defined using `WITH`, gives a name to an intermediate query result that can be referenced in subsequent parts of the same query. In our optimizations, CTEs are used for different purposes, such as calculating averages, pre-aggregating large tables, and filtering relevant records before further processing. Some CTEs use `MATERIALIZED` to ensure that intermediate results are calculated separately and can be reused. CTEs are therefore an implementation technique that supports several optimization strategies rather than a separate optimization category.

The reported before-and-after timings compare a selected single-pass, no-timeout baseline with the mean execution times of three optimized runs. The runs were not identical in all session settings, so the observed differences should not be interpreted as the isolated effect of SQL rewriting under fully controlled conditions.

4.5.1 Avoiding repeated calculations.

Q1, Q6, Q23, Q30, Q41, and Q81 belong to this optimization category. The main objective is to avoid performing the same calculations repeatedly, particularly when using correlated subqueries. This is achieved by calculating intermediate results once and reusing them through CTEs, window functions, or more efficient conditions.

For example, Q1 and Q6 originally used correlated subqueries to calculate average return amounts per store and average item prices per category. We replaced these subqueries with separate CTEs that calculate the averages once and join them to the remaining results. Q23 combines two customer-level store-sales calculations into a single CTE, avoiding repeated aggregation of the same data. It also uses a window function to obtain the maximum four-year sales total. Similarly, Q30 and Q81 replace correlated state-average subqueries with window functions. Finally, Q41 replaces a counting condition with `EXISTS`, which checks whether a matching row exists without explicitly calculating the total number of matches.

Q30 provides a representative example of this optimization. The original query compares each customer's return total with 1.2 times the average return for customers in the same state. This average is calculated using a correlated subquery that references the current customer's state, potentially requiring the same state-level average to be calculated repeatedly. The original condition is shown in Listing 4.1.

```
WHERE ctr1.ctr_total_return > (
    SELECT AVG(ctr_total_return) * 1.2
    FROM customer_total_return ctr2
    WHERE ctr1.ctr_state = ctr2.ctr_state
)
```

Listing 4.1: Q30 before optimization: correlated state-average calculation (excerpt).

To avoid this repeated calculation, we replaced the correlated subquery with a window function. Using `AVG(...) OVER (PARTITION BY return_state)`, the average is calculated separately for each state while keeping the individual customer rows. The calculated average can then be used directly in the final filtering condition, as shown in Listing 4.2.

```
return_with_state_average AS (
    SELECT
        customer_sk,
        return_state,
        total_return,
        AVG(total_return) OVER (
            PARTITION BY return_state
        ) AS state_average_return
    FROM customer_total_return
)

-- Final filtering condition:
AND r.total_return > r.state_average_return * 1.2
```

Listing 4.2: Q30 after optimization: window-based state average (excerpt).

The state average is calculated before joining with the customer table, ensuring that groups with `NULL` customer keys still contribute to the average while being excluded from the final results. The timings for this query are shown in Figure 15. We observe a general improvement for this query before and after optimization.

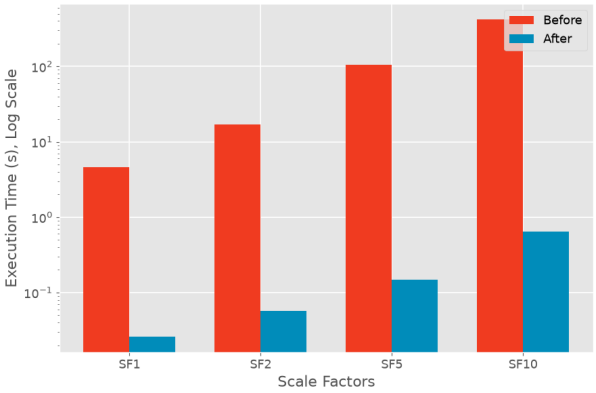


Figure 15: The execution time of Query 30 before and after the optimizations.

Time and space trade-off. In Q30, the window-function rewrite replaces a correlated state-average calculation with a calculation over grouped return records. This can reduce repeated processing, although window evaluation may require sorting or buffering intermediate rows. The actual memory and runtime costs depend on the execution plan and the size of the partitions. The observed runtime improvement does not by itself establish a specific reduction in computational complexity or memory usage.

4.5.2 Combining calculations across periods.

Q4, Q11, Q31, Q74, and Q75 belong to this category. These queries compare values across different years or quarters. In the original versions, values for different periods were often calculated separately and then combined through additional joins. We applied conditional aggregation to calculate values for multiple periods as separate columns within the same grouping operation.

Q4 provides a clear example. The original query calculates annual sales totals for three channels: store, catalog, and web sales. It then references the same `year_total` CTE six times, once for each channel in each of the two years being compared. These references are shown in Listing 4.3.

```
FROM year_total AS t_s_firstyear,
     year_total AS t_s_secyear,
     year_total AS t_c_firstyear,
     year_total AS t_c_secyear,
     year_total AS t_w_firstyear,
     year_total AS t_w_secyear
```

Listing 4.3: Q4 before optimization: six references to annual sales results (excerpt).

To simplify the calculation, we introduced conditional aggregation using `SUM(CASE WHEN ...)` to calculate both yearly totals as separate columns within the same grouping operation. The optimized query creates one summary per sales channel, each containing the values for both years. The store-sales implementation is shown in Listing 4.4.

```
WITH store_agg AS (
    SELECT
        c_customer_id AS customer_id,
        SUM(CASE WHEN d_year = 1998 THEN
            ((ss_ext_list_price - ss_ext_wholesale_cost
              - ss_ext_discount_amt) + ss_ext_sales_price) / 2
```

```
        END) AS s_firstyear,
        SUM(CASE WHEN d_year = 1998 + 1 THEN
            ((ss_ext_list_price - ss_ext_wholesale_cost
              - ss_ext_discount_amt) + ss_ext_sales_price) / 2
        END) AS s_secyear
    FROM customer
    JOIN store_sales
        ON c_customer_sk = ss_customer_sk
    JOIN date_dim
        ON ss_sold_date_sk = d_date_sk
    WHERE d_year IN (1998, 1998 + 1)
    GROUP BY c_customer_id, ...
)
```

Listing 4.4: Q4 after optimization: conditional aggregation for store sales (excerpt).

The optimized query then joins the three channel summaries, each containing both yearly values, instead of joining six references to the original annual summary. The execution times before and after the optimizations can be observed in Figure 16.

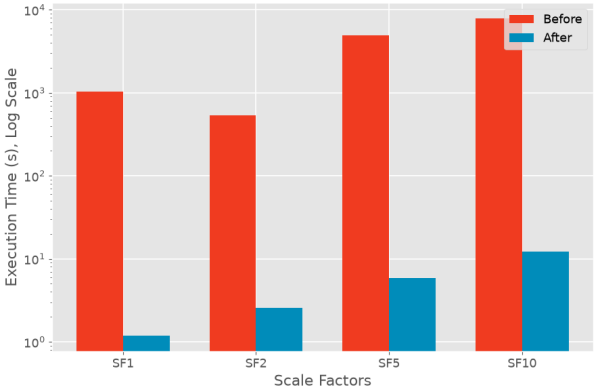


Figure 16: The execution time of Query 4 before and after the optimizations.

The same general technique is applied in the other queries. Q11 uses `SUM(...) FILTER` to calculate yearly store and web sales totals. Q31 uses conditional aggregation to calculate quarterly sales for the first three quarters. Q74 uses `FILTER` with the scale-specific aggregate function (such as `STDDEV_SAMP`, `AVG`, or `MAX`) to calculate both yearly values in one pass. Q75 replaces a self-join between previous-year and current-year sales summaries with conditional aggregation.

Time and space trade-off. In Q04, conditional aggregation computes sales totals for multiple years within the same grouped result, reducing the need to combine multiple references to the original annual summary. Each intermediate group may become wider because it stores several period-specific aggregates, while the number of intermediate rows and subsequent joins may decrease. The actual memory impact depends on the execution plan and data characteristics. Figure 16 shows the measured execution-time improvement, but does not establish lower memory usage.

4.5.3 Pre-aggregating intermediate results.

Q14, Q22, and Q76 belong to this optimization category. These queries reduce repeated processing by creating smaller intermediate summaries before performing subsequent op-

erations. Instead of repeatedly working with detailed records from large fact tables, the required data is aggregated first.

Q14 provides a clear example of this approach. In the original query, the sales tables are accessed multiple times. For example, store sales are used to identify item categories represented across all three sales channels and are read again when calculating the overall sales average. These repeated accesses are illustrated in Listing 4.5.

```sql
-- Store sales used to identify common item categories.
SELECT
    iss.i_brand_id,
    iss.i_class_id,
    iss.i_category_id
FROM store_sales,
     item AS iss,
     date_dim AS d1
WHERE ss_item_sk = iss.i_item_sk
  AND ss_sold_date_sk = d1.d_date_sk
  AND d1.d_year BETWEEN 2000 AND 2000 + 2;

-- Store sales read again for the overall average.
SELECT AVG(quantity * list_price)
FROM (
    SELECT
        ss_quantity AS quantity,
        ss_list_price AS list_price
    FROM store_sales, date_dim
    WHERE ss_sold_date_sk = d_date_sk
      AND d_year BETWEEN 2000 AND 2000 + 2

    UNION ALL
    -- Corresponding catalog and web sales branches.
) AS x
```

Listing 4.5: Q14 before optimization: repeated access to store sales (excerpts).

To reduce this repeated processing, we created materialized CTEs that aggregate the three-year sales data by item for each channel. These smaller summaries are then reused to identify common item categories and calculate the overall average. The following excerpt shows the store-sales summary and its reuse in subsequent calculations.

```sql
ss_3yr AS MATERIALIZED (
    SELECT
        ss_item_sk AS item_sk,
        SUM(ss_quantity * ss_list_price) AS s_sum,
        COUNT(ss_quantity * ss_list_price) AS s_cnt
    FROM store_sales
    JOIN d_3yr ON ss_sold_date_sk = d_date_sk
    GROUP BY ss_item_sk
)

-- Reuse in the common-item calculation:
SELECT ...
FROM ss_3yr AS s
JOIN item AS iss ON s.item_sk = iss.i_item_sk

-- Reuse in the overall average calculation:
SELECT SUM(total_sum) / SUM(total_cnt)
```

```
FROM (
    SELECT SUM(s_sum) AS total_sum,
           SUM(s_cnt) AS total_cnt
    FROM ss_3yr
    UNION ALL
    SELECT SUM(c_sum), SUM(c_cnt) FROM cs_3yr
    UNION ALL
    SELECT SUM(w_sum), SUM(w_cnt) FROM ws_3yr
) AS a
```

Listing 4.6: Q14 after optimization: materialized summaries and their reuse (excerpt).

Equivalent summaries are created for catalog and web sales. The summaries store both the sales sum and the number of non-`NULL` values. This allows the overall average to be calculated correctly by combining the sums and counts across all three channels, rather than averaging individual item averages.

The execution times before and after the optimizations can be observed in Figure 17.

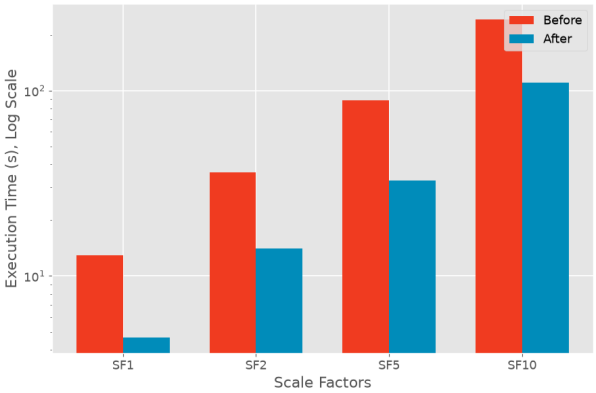


Figure 17: The execution time of Query 14 before and after the optimizations.

The same principle is applied in Q22, where inventory quantities are first aggregated by item using `SUM` and `COUNT` before joining the smaller result with the item table. The average is then calculated from the combined sums and counts. In Q76, each sales channel is aggregated separately before the results are combined using `UNION ALL`.

Time and space trade-off. In Q14, pre-aggregating sales records by item creates intermediate summaries that can be reused in subsequent calculations. This requires space to store the grouped sums and counts, but may reduce repeated processing of detailed fact rows and the amount of data passed to later operations. The benefit depends on the number of distinct grouping keys, the intermediate-result sizes, and the execution plan. The performance results demonstrate runtime differences without directly measuring peak memory usage.

4.5.4 Filtering early and restricting joins.

Q17, Q25, Q29, Q39, and Q95 belong to this category. These queries reduce the amount of data involved in costly joins and aggregations by filtering rows earlier or restricting join inputs to relevant records.

Q17 and Q29 first identify relevant customer–item pairs from matching store-sales and return records. These pairs are then used to restrict the catalog-sales input instead of

joining the entire catalog-sales table with all matching records. This reduces unnecessary matching while retaining the original fact rows for the final calculations.

Q25 follows a similar approach. It first identifies matching store-sales and return records, then restricts catalog sales to the relevant customer–item pairs. It also pre-aggregates catalog data while retaining a row count to preserve the original join multiplicity.

Q39 moves the month filter into the initial inventory query, so only January and February are processed during aggregation instead of calculating statistics for all months and filtering afterward. It also removes the warehouse name from the grouping because the warehouse key already identifies the warehouse.

Q95 provides a clear example of filtering early. The original query identifies orders shipped from multiple warehouses by joining the entire `web_sales` table with itself. This creates pairs of rows belonging to the same order but different warehouses, as shown in Listing 4.7.

```
WITH ws_wh AS (
    SELECT
        ws1.ws_order_number,
        ws1.ws_warehouse_sk AS wh1,
        ws2.ws_warehouse_sk AS wh2
    FROM web_sales AS ws1,
         web_sales AS ws2
    WHERE ws1.ws_order_number = ws2.ws_order_number
      AND ws1.ws_warehouse_sk <> ws2.ws_warehouse_sk
)
```

Listing 4.7: Q95 before optimization: warehouse-pair self-join (excerpt).

To avoid generating warehouse pairs across the entire sales table, we first filter web sales based on the required shipping period, state, and website to identify candidate orders. We then check these orders for shipments from multiple warehouses using `GROUP BY` and `COUNT(DISTINCT ...)`. The optimized implementation is shown in Listing 4.8.

```
WITH filtered_ws AS MATERIALIZED (
    SELECT
        ws1.ws_order_number,
        ws1.ws_ext_ship_cost,
        ws1.ws_net_profit
    FROM web_sales AS ws1
    JOIN date_dim
        ON ws1.ws_ship_date_sk = d_date_sk
    JOIN customer_address
        ON ws1.ws_ship_addr_sk = ca_address_sk
    JOIN web_site
        ON ws1.ws_web_site_sk = web_site_sk
    WHERE d_date BETWEEN '1999-04-01'
                     AND (CAST('1999-04-01' AS date) + 60)
      AND ca_state = 'MT'
      AND web_company_name = 'pri'
),
multi_wh_orders AS (
    SELECT ws_order_number
    FROM web_sales
    WHERE ws_order_number IN (
        SELECT ws_order_number FROM filtered_ws
    )
```

```
    GROUP BY ws_order_number
    HAVING COUNT(DISTINCT ws_warehouse_sk) > 1
)
SELECT ...
FROM filtered_ws
WHERE ws_order_number IN (
    SELECT ws_order_number FROM multi_wh_orders
)
  AND ws_order_number IN (
    SELECT wr_order_number FROM web_returns
)
```

Listing 4.8: Q95 after optimization: early filtering and warehouse counting (excerpt).

Importantly, the second step checks all sales rows belonging to each candidate order, not only the initially filtered rows. This ensures that shipments from other warehouses outside the initial date or address filters are not missed. The optimized query also retains the requirement for a matching web return. The execution times before and after the optimizations can be observed in Figure 18.

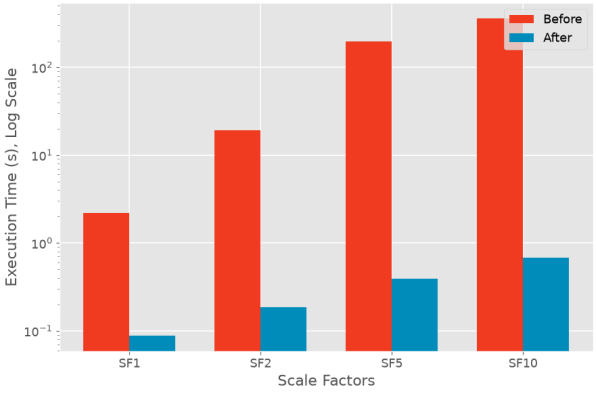


Figure 18: The execution time of Query 95 before and after the optimizations.

Time and space trade-off. In Q95, early filtering materializes a subset of relevant sales records and limits the subsequent warehouse-diversity check to candidate orders. This introduces storage overhead for the filtered intermediate result, but can reduce unnecessary processing compared with the original warehouse self-join. The actual memory and execution-time effects depend on the number of candidate orders, their associated sales rows, and the execution plan. Figure 18 provides runtime evidence but not a direct measurement of memory savings.

4.5.5 Simplifying join syntax and predicates.

Q16 belongs to this category. The query aggregates catalog-sales orders shipped within a 60-day window to a target state and call-center county, requiring that each order includes items shipped from more than one warehouse and has no catalog returns. Unlike Q95, the original Q16 query already uses `EXISTS` and `NOT EXISTS` subqueries rather than an unfiltered fact-table self-join. However, the generated baseline query expresses dimension joins using comma-separated tables in the `FROM` clause, repeats identical county literals five times inside an `IN` list, and uses `SELECT *` inside both existence subqueries, as shown in Listing 4.9.

```
FROM catalog_sales cs1,
     date_dim,
     customer_address,
     call_center
WHERE d_date BETWEEN '2001-3-01'
                 AND (CAST('2001-3-01' AS date) + 60)
  AND cs1.cs_ship_date_sk = d_date_sk
  AND cs1.cs_ship_addr_sk = ca_address_sk
  AND ca_state = 'KY'
  AND cs1.cs_call_center_sk = cc_call_center_sk
  AND cc_county IN (
      'Williamson County', 'Williamson County',
      'Williamson County', 'Williamson County',
      'Williamson County'
  )
  AND EXISTS (
      SELECT *
      FROM catalog_sales cs2
      WHERE cs1.cs_order_number = cs2.cs_order_number
        AND cs1.cs_warehouse_sk <> cs2.cs_warehouse_sk
  )
  AND NOT EXISTS (
      SELECT *
      FROM catalog_returns cr1
      WHERE cs1.cs_order_number = cr1.cr_order_number
  )
```

Listing 4.9: Q16 before optimization: implicit comma joins and repeated county literals (excerpt).

In the optimized version, we replace the implicit comma joins with explicit `JOIN ... ON` clauses, use typed `DATE` literals, replace `SELECT *` with `SELECT 1` inside the `EXISTS` and `NOT EXISTS` subqueries, and deduplicate the county filter values (reducing the five duplicate literals to a single equality comparison at SF1, SF2, and SF5, while keeping the three distinct counties in an `IN` list at SF10). The rewritten query is shown in Listing 4.10.

```
FROM catalog_sales AS cs1
JOIN date_dim AS d
    ON cs1.cs_ship_date_sk = d.d_date_sk
JOIN customer_address AS ca
    ON cs1.cs_ship_addr_sk = ca.ca_address_sk
JOIN call_center AS cc
    ON cs1.cs_call_center_sk = cc.cc_call_center_sk
WHERE d.d_date BETWEEN DATE '2001-03-01'
                   AND DATE '2001-03-01' + 60
  AND ca.ca_state = 'KY'
  AND cc.cc_county = 'Williamson County'
  AND EXISTS (
      SELECT 1
      FROM catalog_sales AS cs2
      WHERE cs1.cs_order_number = cs2.cs_order_number
        AND cs1.cs_warehouse_sk <> cs2.cs_warehouse_sk
  )
  AND NOT EXISTS (
      SELECT 1
      FROM catalog_returns AS cr1
      WHERE cs1.cs_order_number = cr1.cr_order_number
```

```
22      )
```

Listing 4.10: Q16 after optimization: explicit joins, deduplicated predicates, and simplified semi-joins (excerpt).

Retaining `EXISTS` and `NOT EXISTS` allows PostgreSQL to evaluate order membership using semi-joins and anti-joins without multiplying the outer catalog-sales rows.

4.5.6 Queries combining multiple optimization techniques.

Some queries combine techniques from more than one optimization category. Q25 is a good example of combining multiple optimization techniques. The original query directly joins the `store_sales`, `store_returns`, and `catalog_sales` tables using customer and item identifiers, together with the corresponding date conditions. The relevant joins and filters are shown in Listing 4.11.

```
1  FROM store_sales,
2       store_returns,
3       catalog_sales,
4       date_dim d1,
5       date_dim d2,
6       date_dim d3,
7       store,
8       item
9  WHERE d1.d_moy = 4
10   AND d1.d_year = 2002
11   AND d1.d_date_sk = ss_sold_date_sk
12   AND i_item_sk = ss_item_sk
13   AND s_store_sk = ss_store_sk
14   AND ss_customer_sk = sr_customer_sk
15   AND ss_item_sk = sr_item_sk
16   AND ss_ticket_number = sr_ticket_number
17   AND sr_returned_date_sk = d2.d_date_sk
18   AND d2.d_moy BETWEEN 4 AND 10
19   AND d2.d_year = 2002
20   AND sr_customer_sk = cs_bill_customer_sk
21   AND sr_item_sk = cs_item_sk
22   AND cs_sold_date_sk = d3.d_date_sk
23   AND d3.d_moy BETWEEN 4 AND 10
24   AND d3.d_year = 2002
```

Listing 4.11: Q25 at SF10 before optimization: joining detailed sales and return records (excerpt).

This approach combines the detailed records from all three fact tables before the final aggregation. As a result, multiple catalog-sales records can match the same customer–item pair, increasing the number of intermediate rows and repeating the corresponding store-sales and return values.

To address this, we applied two optimizations. First, we materialized the matching store-sales and return records after applying their date filters. These records are then used to identify the relevant customer–item pairs. Second, we pre-aggregate catalog sales by customer and item, using `EXISTS` to exclude records that do not match the filtered store-sales and return results. The optimized implementation is shown in Listing 4.12.

```
1  WITH matched_store_returns AS MATERIALIZED (
2      SELECT
```

```
        ss.ss_item_sk AS item_sk,
        ss.ss_store_sk AS store_sk,
        ss.ss_customer_sk AS customer_sk,
        ss.ss_net_profit AS store_profit,
        sr.sr_net_loss AS return_loss
    FROM store_sales AS ss
    JOIN date_dim AS sold_date
        ON sold_date.d_date_sk = ss.ss_sold_date_sk
    JOIN store_returns AS sr
        ON sr.sr_customer_sk = ss.ss_customer_sk
       AND sr.sr_item_sk = ss.ss_item_sk
       AND sr.sr_ticket_number = ss.ss_ticket_number
    JOIN date_dim AS return_date
        ON return_date.d_date_sk = sr.sr_returned_date_sk
    WHERE sold_date.d_year = 2002
      AND sold_date.d_moy = 4
      AND return_date.d_year = 2002
      AND return_date.d_moy BETWEEN 4 AND 10
),
catalog_profit AS (
    SELECT
        cs.cs_bill_customer_sk AS customer_sk,
        cs.cs_item_sk AS item_sk,
        COUNT(*) AS catalog_row_count,
        SUM(cs.cs_net_profit) AS catalog_profit_sum
    FROM catalog_sales AS cs
    JOIN date_dim AS sold_date
        ON sold_date.d_date_sk = cs.cs_sold_date_sk
    WHERE sold_date.d_year = 2002
      AND sold_date.d_moy BETWEEN 4 AND 10
      AND EXISTS (
          SELECT 1
          FROM matched_store_returns AS m
          WHERE m.customer_sk = cs.cs_bill_customer_sk
            AND m.item_sk = cs.cs_item_sk
      )
    GROUP BY
        cs.cs_bill_customer_sk,
        cs.cs_item_sk
)
SELECT
    i.i_item_id,
    i.i_item_desc,
    s.s_store_id,
    s.s_store_name,
    SUM(m.store_profit * c.catalog_row_count)
        AS store_sales_profit,
    SUM(m.return_loss * c.catalog_row_count)
        AS store_returns_loss,
    SUM(c.catalog_profit_sum)
        AS catalog_sales_profit
FROM matched_store_returns AS m
JOIN catalog_profit AS c
    ON c.customer_sk = m.customer_sk
   AND c.item_sk = m.item_sk
JOIN item AS i
    ON i.i_item_sk = m.item_sk
JOIN store AS s
```

```
    ON s.s_store_sk = m.store_sk
GROUP BY
    i.i_item_id,
    i.i_item_desc,
    s.s_store_id,
    s.s_store_name
ORDER BY
    i.i_item_id,
    i.i_item_desc,
    s.s_store_id,
    s.s_store_name
LIMIT 100;
```

Listing 4.12: Q25 at SF10 after optimization: early filtering and catalog pre-aggregation (excerpt).

An important detail is that the catalog summary retains both the total profit and the number of matching rows. The row count is used to multiply the store-sales and return values, preserving the original join multiplicity. This ensures that the pre-aggregation does not change the final results while reducing the amount of detailed catalog data processed. The execution times before and after the optimizations can be observed in Figure 19.

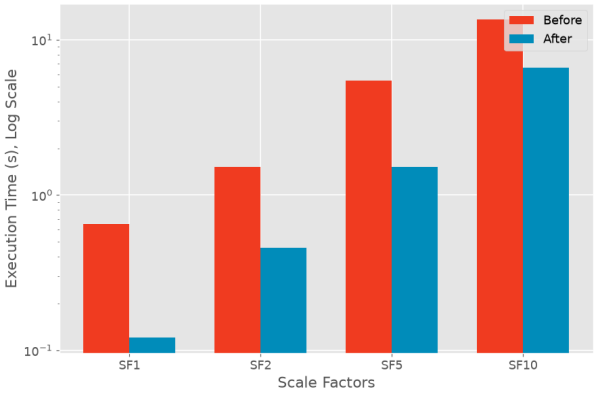


Figure 19: The execution time of Query 25 before and after the optimizations.

Time and space trade-off. In the SF10 version of Q25, materializing the matching store-sales and return records and pre-aggregating catalog sales introduces intermediate storage for the filtered records and grouped catalog summaries. The catalog row count helps preserve the multiplicity of the original joins while avoiding repeated processing of individual catalog rows in the final aggregation. This may reduce intermediate join work, although the overall memory and runtime effects depend on the number of matches, the grouping cardinality, and the execution plan.

4.6 Indexes

After optimizing the bottleneck queries, we investigated whether the physical design adds further benefit. All comparisons in this section use the same optimized 99-query workload on the same data, in three configurations (Section 3.3): optimized queries without secondary indexes, with the 8 B-tree indexes selected with Dexter and HypoPG, and with the same indexes plus clustering of the four fact tables. Table 9 summarizes

the total execution time of the 99 queries; the following subsections discuss the indexes and the clustering in turn.

	Optimized	+ indexes		+ clustering	
SF	(s)	(s)	change	(s)	change
1	23.5	19.1	−18.7%	21.1	−10.2%
2	58.4	49.2	−15.8%	43.0	−26.4%
5	180.2	168.5	−6.5%	171.5	−4.8%
10	536.7	492.3	−8.3%	416.9	−22.3%

Table 9: Total execution time of the 99 optimized queries per physical configuration (change relative to optimized queries without indexes).

4.6.1 Secondary Indexes

Figure 20 shows the execution times of the nine identified outlier queries at each scale factor before optimization, after query optimization, and after optimization with indexes. Adding indexes after query optimization does not always improve performance, and for some queries it even increases the execution time. This is expected for a decision-support workload: most TPC-DS queries aggregate large parts of the fact tables, so the planner keeps a sequential scan with a hash join and an index brings no benefit, and where the planner does choose an index plan it can cost more than the scan it replaces.

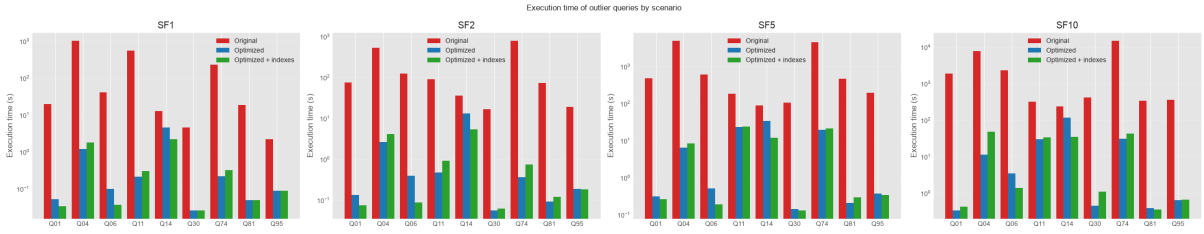


Figure 20: Execution time of the top 9 outlier queries before optimization, after optimization, and after optimization with indexes.

The total execution time of the 99 queries nevertheless decreases at every scale factor (Table 9): by 18.7% at SF1, 15.8% at SF2, 6.5% at SF5 and 8.3% at SF10. The gains on the expensive queries are therefore larger than the regressions on the others. The indexes are not free, however: they lengthen the load (Section 4.1). At SF10 they save 44.4 s per run of the 99 queries, so they only pay off if the workload is run often enough to make up for the extra load time.

4.6.2 Clustering

In PostgreSQL, `CLUSTER` rewrites a table once in the order of an index. We applied it to the four fact tables using their date-based indexes (Listing 3.1), so rows with nearby dates are stored on nearby pages. Date-range filters and date joins then read contiguous pages instead of pages scattered over the table, which reduces random I/O.

Figure 21 compares the total execution times across the four scale factors under the three configurations: optimized queries, optimized queries with indexes, and optimized

queries with indexes and clustering. As expected, execution time grows with the scale factor in all configurations. Clustering reduces the total time compared with the optimized queries alone at every scale factor, by between 4.8% (SF5) and 26.4% (SF2). Compared with indexes alone, it gives the lowest total time at SF2 and SF10. The clearest gain is at SF10, where the total decreases from 536.7 s for the optimized queries to 492.3 s with indexes and 416.9 s with clustering, a reduction of 22.3% compared with optimization alone and 15.3% compared with indexes alone. At SF1 and SF5 the two physical designs are close, with indexes alone slightly ahead (19.1 s against 21.1 s at SF1 and 168.5 s against 171.5 s at SF5).

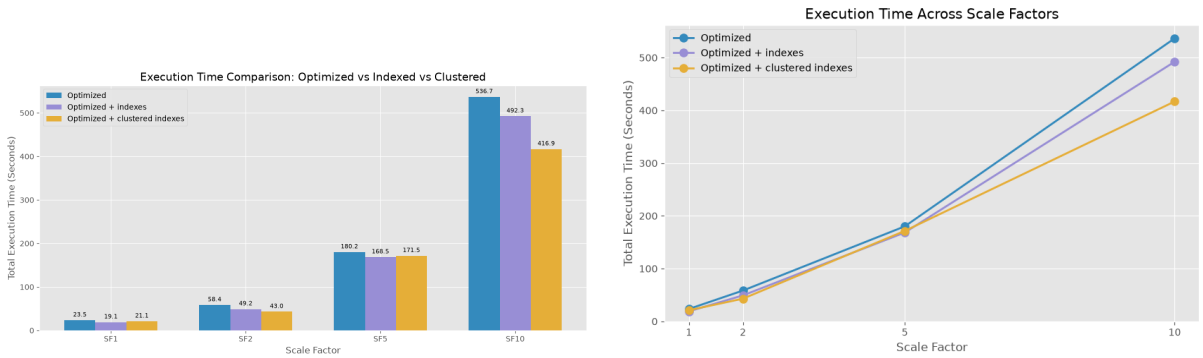


Figure 21: Total query execution time of 99 queries across scale factors (SF1–SF10) for three configurations: optimized queries, optimized queries with indexes, and optimized queries with indexes and clustering, illustrating execution time growth as data volume increases.

5 Conclusion and Future Work

5.1 Conclusion

In this project, we set up and automated a TPC-DS-derived benchmark on PostgreSQL 16.15 inside Docker to study how analytical queries scale and how SQL query rewriting, secondary indexing, and physical clustering affect performance across SF $\in \{1, 2, 5, 10\}$. Our main conclusions are as follows:

1. **Scaling Behavior Across Scale Factors:** Across the four scale factors, the overall execution time of the 99-query workload increases with data volume, following a clear upward scaling pattern across configurations as dataset size grows from SF = 1 to SF = 10. At the individual query level, most baseline queries scale approximately linearly with data size, whereas a smaller fraction follows exponential growth trajectories or exhibits irregular runtime variations due to changes in query parameters, data distribution, and filter selectivity.

2. **Main Causes of Slow Queries:** Most of the baseline execution time was concentrated in a small group of outlier queries. Execution plan inspection showed that the worst slowdowns came from four recurring patterns: multi-way self-joins over materialized CTEs that led the planner to choose nested loops (such as Q4, Q11, and Q74), correlated subqueries rescanning a CTE for every outer row (such as Q1, Q6, Q30, and Q81), repeated multi-year scans of the same fact tables (Q14), and unfiltered self-joins on large fact tables (Q95).

3. **Effect of SQL Query Optimization:** Rewriting the 20 slowest queries across all four scale factors produced the largest performance improvement. By replacing multi-way self-joins with conditional aggregation (`FILTER` and `CASE WHEN`), turning correlated subqueries into window functions (`OVER (PARTITION BY ...)`) or grouped joins, and filtering or pre-aggregating candidate keys early while preserving exact duplicate and `NULL` semantics, the reported SF10 workload total dropped by 98.2% (from 29,048.8 s in the no-timeout baseline pass to 536.7 s in the rewritten suite).

4. **Effect of Indexing and Clustering:** Adding the eight secondary B-Tree indexes selected with `Dexter` and `HypoPG` reduced the total runtime of the optimized workload by an additional 6.5% to 18.7% across scale factors, despite minor regressions on a few individual queries. Physically reordering the four fact tables with `CLUSTER` on their date indexes achieved the lowest total runtime at SF = 2 (43.0 s) and SF = 10 (416.9 s, or 22.3% faster than query optimization alone).

5.2 Future Work

Several extensions could build on this study:

- **Extended Optimizer Statistics:** Using PostgreSQL's `CREATE STATISTICS` command to capture multivariate column correlations could improve row-count estimates on multi-column filters without manual query rewrites.

- **Throughput and Data Maintenance Tests:** Extending the benchmark runner to execute the multi-stream Throughput Test and the Data Maintenance Test, as well as evaluating columnar storage extensions at larger scale factors, would cover the remaining components of the TPC-DS specification.

5.3 LLM Usage

During this project, we used Large Language Models (LLMs) as supplementary tools to support different stages of the development and documentation process. The tools used included ChatGPT, Claude, and Google Gemini. Their purpose was to assist with technical tasks and improve the efficiency of our work, rather than replace the understanding and decisions of the project authors.

The main applications of these tools throughout the project were:

- **Technical Understanding and Research:** LLMs were used to clarify unfamiliar technical concepts related to database systems, PostgreSQL, the TPC-DS benchmark, and SQL query optimization. They also supported initial discussions and exploration of different technical approaches. Relevant information was checked against official documentation and other sources where applicable.

- **Code Development and Debugging:** We used LLMs to assist with developing, reviewing, and debugging Python scripts used in the experimental workflow. They were also used to discuss SQL query structures and explore possible optimization strategies.

- **Report Preparation:** LLMs were used to improve the clarity and readability of parts of this report, assist with organizing technical explanations, and convert or adjust text into the required LaTeX format. All generated or revised content was reviewed and edited by the authors before being included in the final report.

Overall, LLMs served as supporting tools throughout the project. The project structure, experimental methodology, configuration of the benchmarking environment, evaluation of the results, and final conclusions were determined and controlled by the project authors.

References

[1] Gianni Ciolli and Simon Riggs. *PostgreSQL 14 Administration Cookbook*. Packt Publishing, 2022.

[2] PostgreSQL Documentation. Postgresql global development group. *Poveznica: https://www. postgresql. org/docs*, 2006.

[3] Neil Matthew and Richard Stones. *Beginning databases with postgreSQL: From novice to professional*. Springer, 2005.

[4] Raghunath Othayoth Nambiar and Meikel Poess. The making of tpc-ds. In *VLDB*, volume 6, pages 1049–1058, 2006.

[5] Regina O Obe and Leo S Hsu. *PostgreSQL: up and running*. " O'Reilly Media, Inc.", 2014.

[6] Meikel Poess, Raghunath Othayoth Nambiar, and David Walrath. Why you should run tpc-ds: A workload analysis. In *VLDB*, volume 7, pages 1138–1149, 2007.

[7] scikit-learn developers. Metrics and scoring: Quantifying the quality of predictions. `https://scikit-learn.org/stable/modules/model_evaluation.html#r2-score`. Accessed: 2026-11-07.

[8] Michael Stonebraker and Lawrence A Rowe. The design of postgres. *ACM Sigmod Record*, 15(2):340–355, 1986.

[9] Transaction Processing Performance Council. TPC Benchmark DS Standard Specification, Version 4.0.0. Technical report, Transaction Processing Performance Council, November 2024.

[10] Efraim Turban. *Decision support and expert systems: Managerial perspectives*. Macmillan Library Reference, 1990.

[11] Alejandro Vaisman and Esteban Zimányi. *Data warehouse systems: design and implementation*, volume 625. Springer, Heidelberg, 2014.